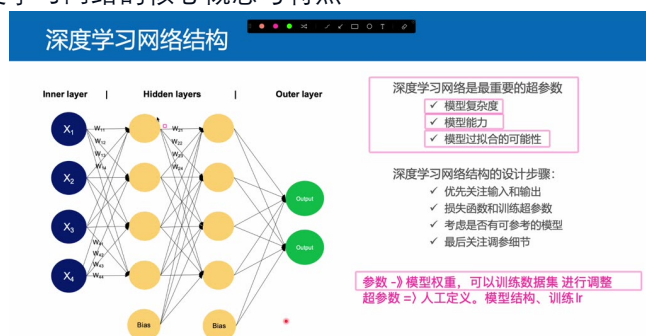


一、课程主要内容 18:50

1. 深度学习网络结构 19:56

1) 深度学习网络 20:15

● 深度学习网络的核心概念与特点



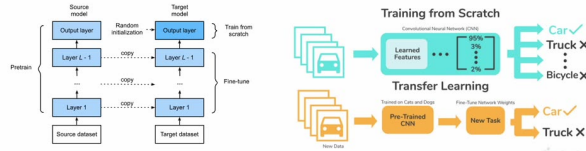
-
- **核心概念:** 深度学习通过多层网络, 逐层进行计算, 最终得到输出结果。
- **与浅层学习的区别:** 深度学习 (如神经网络) 与浅层学习 (如传统机器学习模型, 如sklearn中的模型) 相对, 后者参数较少, 结构简单。
- **关键特点:** 深度学习模型拥有非常多的参数, 这是其强大能力的基础。
- **参数与超参数的定义与区别 21:20**
 - **参数 (模型权重):** 指模型中的所有权重 (如 w_1, w_2 等), 这些权重可以在训练过程中, 结合训练数据集自动进行调整。
 - **超参数:** 由人工预先定义, 不能通过训练自动调整。
 - **常见超参数举例:** 模型结构 (如层数、每层神经元数量)、训练轮数、学习率 (lr) 等。
 - **实际案例:** 千问模型的整体架构 (如层数、损失函数、学习率设置) 就是其超参数。
- **超参数对模型的影响 22:34**
 - **超参数的重要性:** 在深度网络中, 超参数是最重要的, 它直接决定了模型的最终表现。
 - **具体影响方面:**
 - **模型复杂度:** 超参数决定了模型的复杂程度。
 - **模型能力:** 影响模型的学习和表达能力。
 - **过拟合可能性:** 不合理的超参数设置会增加模型过拟合的风险。
- **深度学习网络结构的设计步骤 23:30**
 - **设计步骤:**
 - **优先关注输入和输出:** 明确模型的输入数据和期望的输出结果。
 - **确定损失函数和训练超参数:** 选择合适的损失函数, 并设定学习率等训练超参数。
 - **考虑是否有可参考的模型:** 优先寻找已有的成熟模型进行复用或修改。
 - **最后关注调参细节:** 在模型框架确定后, 再进行细致的参数调整。
- 2) 迁移学习 24:01
- 迁移学习的定义与作用 24:05

深度学习与迁移学习

迁移学习是一种将一个任务中学习到的知识，应用到另一个相关任务上的方法。它旨在解决深度学习的训练难题。

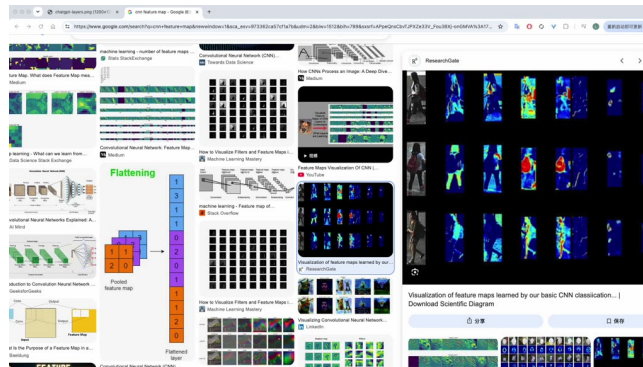
✓ **特征提取器**：保留预训练模型的大部分层，只用它来提取新任务数据的特征。

✓ **微调 (Fine-tuning)**：在特征提取的基础上，对模型的部分甚至全部参数进行微调，使其更好地适应新任务。



-
- **定义**：迁移学习是一种将一个任务（源任务）中学习到的知识，应用到另一个相关任务（目标任务）上的方法。
- **核心目的**：解决深度学习训练中的难题，如数据不足、训练耗时、资源浪费等。
- **核心思想**：复用已有模型，避免从头开始训练。
 - **举例说明**：如果两个图像分类任务非常相似，从头训练两个模型非常耗时且浪费资源。迁移学习允许我们复用第一个任务的模型来训练第二个任务。

● 逐层特征提取与模型复用 29:30



-
- **深度学习模型的层次化特征提取**：
 - **浅层（靠近输入）**：提取底层特征，如边缘、线条、形状等。
 - **深层（靠近输出）**：提取高层语义信息，如物体、场景等。
- **模型复用的原理**：对于相似的任务（如图像分类），模型的前面层（特征提取层）提取的特征是通用的，可以直接照搬复用。
 - **复用方式**：将源模型的网络结构和权重（参数）直接复制（copy）过来。
 - **输出层调整**：由于目标任务可能不同（如分类类别数不同），需要重新定义输出层，并随机初始化其参数。

● 特征提取器与微调 30:34

- **特征提取器**：保留预训练模型的大部分层（尤其是前面的特征提取层），仅用它们来提取新任务数据的特征，然后训练一个新的分类器。
- **微调 (Fine-tuning)**：在特征提取的基础上，对模型的部分甚至全部参数进行进一步训练。
 - **过程**：将源模型的参数作为目标模型的初始化参数，然后在新的任务数据上继续训练，使模型更好地适应新任务。
 - **与从头训练的区别**：微调不是从头开始训练，而是在已有参数的基础上进行“微调”，因此训练速度更快，所需数据更少。

● 迁移学习的应用与重要性 33:00

- **广泛应用**：迁移学习是深度学习得以广泛应用的重要原因之一。
- **现代应用案例**：大语言模型的微调（Fine-tuning）就是迁移学习的典型应用。例如，对千问模型进行微调，使其适应特定领域的任务，而不是从头训练一个全新的模型，这非常不划算。

3) 深度学习调参 33:26

● 深度学习调参概述与非凸优化问题 33:29

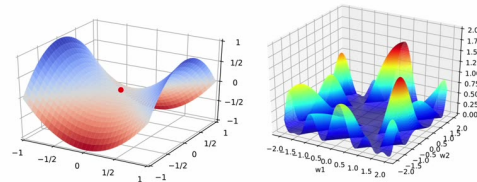
深度学习调参

非凸优化问题

- 参数初始化
- 逃离局部最优

高维的非凸优化

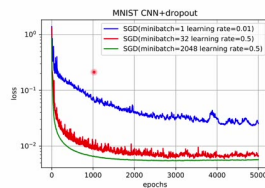
- 鞍点
- 平摊底部



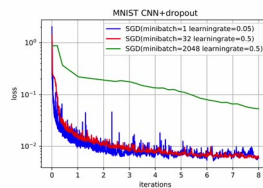
- - 深度学习本质: 深度学习模型的训练过程是一个非凸优化问题。
 - 局部最优困境: 由于是非凸问题, 模型在参数优化时, 往往只能找到局部最优解, 很难找到全局最优解。
 - 调参目的: 模型调参 (调整超参数) 就是为了尽可能让模型跳出局部最优, 逼近更好的解。
 - 常见难点: 高维空间中的非凸优化常遇到鞍点和平摊底部, 导致模型难以继续优化。
- Epoch, Batch, Learning Rate 34:28

深度学习调参

Epoch, Batch, Learning Rate



(a) 按每次小批量更新的损失变化



(b) 按整个数据集迭代的损失变化

- - 核心超参数: 模型调参主要调整三个核心超参数: **Epoch**、**Batch**、**Learning Rate**。
 - **Epoch (迭代轮次)**: 指将整个训练数据集从头到尾完整训练一遍的次数。
 - **Batch (批次)**: 将数据集划分为多个小批量样本, 每个小批量称为一个Batch。
 - 举例: 若数据集有1000个样本, 每10个样本划分为一个Batch, 则共有100个Batch。
 - **Batch Size**: 每个Batch中包含的样本数量 (上例中为10)。
 - **Learning Rate (学习率)**: 控制模型参数更新的幅度。
 - 作用: 学习率越大, 模型参数调整的速度越快。
 - **Loss (损失)**: 模型输出结果与真实标签之间的差异。
 - 意义: Loss越低, 代表模型预测越准确。Loss通常是一个非零的正数。
 - **Loss曲线**: 通过设置不同的超参数, 可以绘制Loss随训练次数 (Epoch) 变化的曲线, 用于对比不同超参数的效果。
- 学习率与优化器 38:08

深度学习调参

学习率决定了网络优化速度，也决定了跳出局部最优的可能性。

- 动态学习率
- 自适应学习率

标准的（小批量）梯度下降

学习率

✓ 学习率衰减

✓ Adagrad

✓ Adadelta

✓ RMSprop

梯度

✓ Momentum

✓ Nesterov accelerated gradient

✓ 梯度截断

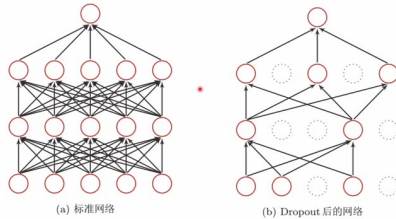
$$\Delta \theta_t = -\frac{\alpha}{\sqrt{G_t + \epsilon}} \odot g_t$$

- - **学习率类型:**
 - **静态学习率:** 训练过程中学习率保持不变。
 - **动态学习率:** 训练过程中学习率会动态调整，如**学习率衰减**。
 - **优化器:** 学习率需要搭配优化器使用。
 - **常见优化器:** Adam、AdamW、带动量的梯度下降（Momentum）等。
 - **自适应学习率:** 如Adagrad、Adadelta、RMSprop等，能自动调整每个参数的学习率。
 - **梯度截断:** 防止梯度爆炸，将梯度限制在一定范围内。
- Dropout与模型微调 38:50

深度学习调参

Dropout

- 集成学习: 原始网络可以近似看作不同子网络的组合模型。
- 贝叶斯学习: 参数 θ 为随机向量，并且先验分布为 $q(\theta)$



- - **Dropout定义:** 在训练过程中，随机“关闭”网络中的一部分节点（神经元），使网络变成不完整的结构。
 - **核心原理:**
 - **随机性:** 每次训练时，被关闭的节点是随机选择的（例如按30%的比例关闭）。
 - **集成学习视角:** 可以看作是将原始网络近似为不同子网络的组合模型。
 - **贝叶斯学习视角:** 将参数视为随机向量，并带有先验分布。
 - **训练与推理的区别:**
 - **训练时:** 使用“残缺”的网络进行训练，增强模型的鲁棒性。
 - **推理时:** 使用完整的网络进行预测。
 - **学习场景:**
 - **场景一（微调）:** 对千问、DeepSeek等模型进行本地微调时，需要理解这些底层原理。
 - **场景二（面试）:** 面试官可能会询问深度学习训练过程、Dropout原理或模型结构。
- Pytorch使用案例 39:03

Pytorch使用案例



-
- **模型定义:** 通过继承nn.Module类定义网络结构（如PyTorchNet）。
 - **网络层:** 使用nn.Linear定义全连接层，指定输入和输出维度。
 - **激活函数:** 使用torch.relu作为激活函数。
 - **前向传播:** 在forward方法中定义数据流动顺序。
- **优化器定义:** 使用optim.Adam定义优化器，并传入模型参数和学习率（lr=0.001）。
- **损失函数定义:** 使用nn.CrossEntropyLoss定义交叉熵损失函数（常用于分类任务）。
- **调参经验与超参数优化方法 43:26**

深度学习调参

调参经验	超参数	优化方法
✓ 用 ReLU 作为激活函数	层数	网格搜索
✓ 分类时用交叉熵作为损失函数	每层神经元个数	随机搜索
✓ SDG+mini-batch	激活函数	贝叶斯优化
✓ 每次迭代都重新随机排序	学习率（以及动态调整算法）	动态资源分配
✓ 数据预处理（标准归一化）	正则化系数	神经架构搜索
✓ 动态学习率（越来越小）	mini-batch 大小	
✓ 用 L1 或 L2 正则化（跳过前几轮）		
✓ 逐层归一化		
✓ dropout		
✓ 数据增强		

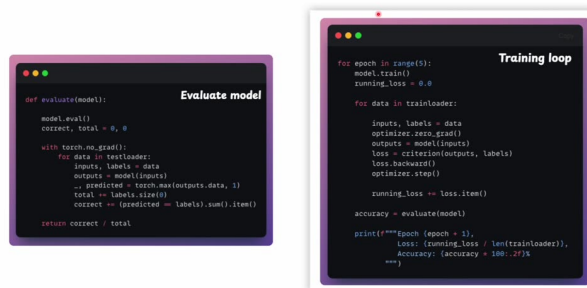
- - **常用优化方法:**
 - **网格搜索:** 遍历所有超参数组合。
 - **随机搜索:** 随机选择超参数组合。
 - **贝叶斯优化:** 基于历史结果，智能选择下一组超参数。
 - **动态资源分配:** 根据训练效果动态分配计算资源。
 - **神经架构搜索:** 自动搜索最优的网络结构。
 - **调参经验总结:**
 - **激活函数:** 优先使用ReLU。
 - **损失函数:** 分类任务使用交叉熵损失函数。
 - **优化器:** 使用SGD + mini-batch。
 - **数据预处理:** 进行标准归一化。
 - **学习率:** 使用动态学习率（越来越小）。
 - **正则化:** 使用L1或L2正则化（可跳过前几轮）。
 - **Dropout:** 使用Dropout防止过拟合。
 - **数据增强:** 增加训练数据多样性。
 - **训练技巧:** 每次迭代都重新随机排序数据。
 - **人工调参为主:** 虽然程序可以自动调参，但目前人工调参仍然更加方便和常用。
- 4) PyTorch使用案例 44:01
- **PyTorch模型定义、优化器及损失函数**

Pytorch使用案例



-
- **PyTorch的本质:** 深度学习框架/库，用于将深度学习概念转化为可执行的代码。
 - 掌握PyTorch对本地RAG应用、复杂Agent应用、模型微调等场景至关重要。
- **模型定义核心步骤:**
 - **继承基类:** 自定义模型需继承 `nn.Module` 基类。
 - **实现两个关键函数:**
 - `__init__` (初始化): 定义网络层结构 (如全连接层 `nn.Linear`) 。
 - `forward` (前向传播): 定义层与层之间的连接方式和计算逻辑 (如激活函数 `torch.relu`) 。
- **模型使用三要素:**
 - **实例化模型:** `model = PyTorchNet()`
 - **定义优化器:** 决定如何调整模型参数，如 `optim.Adam(model.parameters(), lr=0.001)`。
 - `lr=0.001` 是学习率，控制参数更新幅度。
 - **定义损失函数:** 衡量模型输出与真实标签的差异，如 `nn.CrossEntropyLoss()` (交叉熵损失) 。
- 例题1:PyTorch模型训练与验证过程 48:10

Pytorch使用案例



-
- **训练过程核心结构:** 双重for循环。
 - **外层循环 (Epoch):** 将整个数据集完整迭代一次。
 - **内层循环 (Batch):** 每次处理一小批数据。
- **训练步骤详解:**
 - **设置训练模式:** `model.train()`，影响Dropout和Batch Normalization层的行为，不改变网络结构。
 - **获取数据:** 从数据加载器 `trainloader` 中获取 `inputs` (输入) 和 `labels` (真实标签) 。
 - **清空梯度:** `optimizer.zero_grad()`，PyTorch默认不会自动清空梯度，需手动操作。
 - **正向传播:** `outputs = model(inputs)`，从输入逐层计算得到输出。
 - **计算损失:** `loss = criterion(outputs, labels)`，计算输出与标签的差异。
 - **反向传播:** `loss.backward()`，计算每个模型参数的梯度 (偏导数) 。

- **更新参数:** `optimizer.step()`, 根据梯度更新模型权重。
 - **验证过程特点:**
 - **设置验证模式:** `model.eval()`。
 - **关闭梯度计算:** 使用 `with torch.no_grad():` 包裹, 不记录梯度, 不更新参数。
 - **仅做正向传播:** 计算输出并统计准确率, 无需反向传播。
 - **学习建议:** 这部分代码是固定语法, 多写几遍即可掌握流程, 也可借助大模型或 Web Coding 生成。
- **例题2: PyTorch 在各领域的应用案例 53:50**

Pytorch使用案例

案例1: 使用用户的交易记录、账户信息等, 将交易金额、时间、地点、商户类型、历史交易行为等作为输入特征, 通过多层非线性变换来学习这些特征之间的复杂模式, 最终输出一个表示该交易是欺诈行为的概率。

案例2: 各种交通标志的图像数据集, 如停车标志、限速标志、转向标志等。需要设计模型用于自动驾驶汽车识别道路上的交通标志, 以便做出正确的驾驶决策。

案例3: 人脸图像包含对应关键点坐标, 如眼睛、眉毛、鼻子、嘴巴和脸部轮廓的关键点。通过精准定位人脸上的关键点, 以便在这些点上叠加虚拟的化妆效果, 如口红、眼影、眉形等, 实现虚拟试妆功能。

-
- **案例1: 金融风控 - 欺诈交易检测**
 - **输入特征:** 交易金额、时间、地点、商户类型、历史交易行为等。
 - **输出:** 该交易是欺诈行为的概率。
 - **原理:** 通过多层非线性变换学习特征间的复杂模式。
- **案例2: 自动驾驶 - 交通标志识别**
 - **输入:** 交通标志图像数据集 (停车、限速、转向等)。
 - **输出:** 交通标志类型。
 - **应用:** 辅助自动驾驶汽车做出正确驾驶决策。
- **案例3: 虚拟试妆 - 人脸关键点定位**
 - **输入:** 人脸图像。
 - **输出:** 眼睛、眉毛、鼻子、嘴巴、脸部轮廓等关键点坐标。
 - **应用:** 精确定位后叠加虚拟化妆效果 (口红、眼影、眉形等), 如抖音底层特效。
- **深度学习与机器学习的关系 55:09**
 - **深度学习是机器学习的子集:** 深度学习是机器学习中的一个重要分支。
 - **应用范围:** 深度学习既可做判别模型 (如分类、回归), 也可做生成模型 (如图像生成、文本生成)。

5) 判别模型与生成模型 55:24

- **判别模型的定义与使用场景 56:09**

判别模型与生成模型



判别模型: 模型输出的结果更加确定, 数据通过模型逐步降低
生成模型: 模型从原始数据分布中进行采样, 生成新的样本

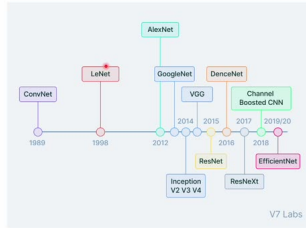
○

- **核心定义:** 判别模型 (Discriminative Model) 的核心任务是“做判断”, 本质是一个逐步降维的过程。
 - **降维过程:** 输入数据 (如一张图片) 维度很高 (例如 $24 \times 24 \times 3$), 模型通过层层处理, 最终输出一个低维度的结果 (如0或1的分类标签)。
 - **输出特点:** 模型输出的结果非常确定, 直接给出判断结论。
- **典型使用场景:**
 - **文本分类:** 判断一封邮件是否为垃圾邮件。
 - **法律条文分类:** 判断一条法律条文属于哪种具体类型。
 - **意图识别:** 判断用户的输入属于哪一种意图。
- **关键词:** Discriminative Model、降维、判断、分类。
- **生成模型的定义与特点 57:24**
 - **核心定义:** 生成模型 (Generative Model) 的核心任务是“生成样本”, 其维度没有降低。
 - **学习过程:** 模型在训练时看到大量样本 (如很多马的图片), 从中学习到原始数据的分布规律。
 - **生成过程:** 训练完成后, 模型从学习到的数据分布中进行随机采样, 逐步生成全新的、与训练集相似的样本。
 - **与判别模型的本质区别:**
 - **维度变化:** 判别模型是降维, 生成模型维度不变或升高 (生成新数据)。
 - **最终目标:** 判别模型输出一个标签或判断; 生成模型输出一个完整的样本 (如图片、文本)。
 - **关键词:** Generative Model、采样、生成样本、数据分布。
- **生成模型的典型例子 59:15**
 - **大语言模型 (LLM):** 本质就是生成模型。
 - **典型代表:** ChatGPT、通义千问、DeepSeek等。
 - **能力体现:** 可以源源不断地生成新的文本样本。
 - **图像生成模型:** 训练集包含大量人脸图片, 训练后模型可以不断生成新的人脸图像。
 - **GPT中的“G”:** GPT (Generative Pre-trained Transformer) 中的“G”指的就是生成模型。
- 6) 传统深度学习解决方案范式 01:00:01
 - **大模型时代之前:** 针对不同任务, 需要单独设计不同的网络结构。
 - **典型例子:** 专门用于图像分类的网络、专门用于语音识别的网络、专门用于文本翻译的网络。
 - **模型类型:** 这些单独设计的网络结构, 本质上基本都是判别模型。
 - **大模型时代之后:** 可以将所有单独设计的网络结构收敛到一个统一的生成模型中。
 - **统一模型:** 这个统一的生成模型就是现在的大模型 (如通义千问)。
 - **核心优势:** 一个大模型可以完完全全替代之前所有单独设计的网络结构, 实现“一统天下”。
- 2. 休息与课程回顾 01:01:25
 - 1) 传统深度学习解决方案范式

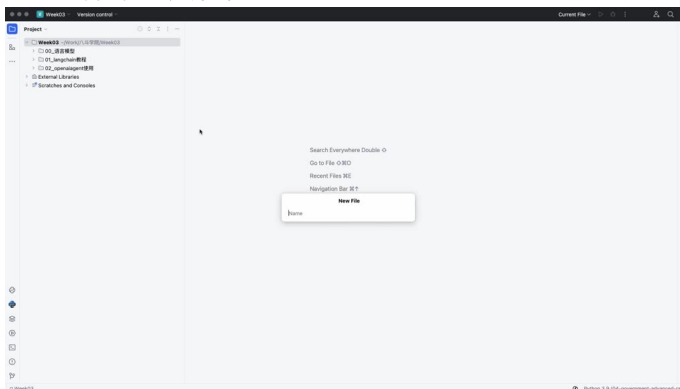
传统的深度学习解决方案范式

针对不同的任务设计不同的网络:

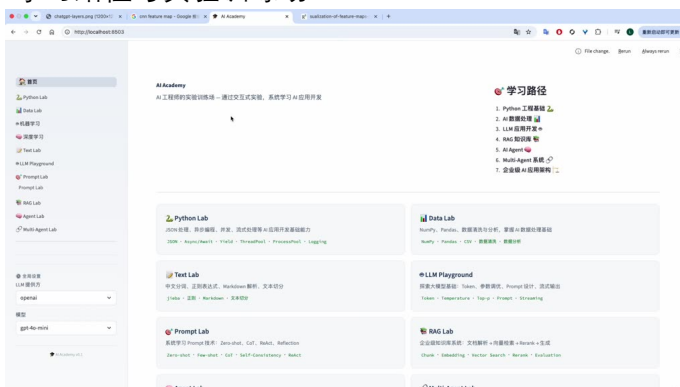
- ✓ 用于图像分类的网络
- ✓ 用于文本分类的网络
- ✓ 用于实体抽取的网络
- ✓ 用户文本翻译的网络
- ✓ ...



- - **核心思路:** 针对不同的任务，需要设计不同的网络结构。
 - **典型例子:**
 - 图像分类 → 使用卷积神经网络（CNN）。
 - 文本分类 → 使用循环神经网络（RNN）或Transformer。
 - 实体抽取 → 使用序列标注模型。
 - 文本翻译 → 使用Seq2Seq模型。
 - **网络发展历程:** 从早期的LeNet、AlexNet，到后来的VGG、GoogleNet、ResNet、EfficientNet等，网络结构不断演进，性能持续提升。
- 2) 休息与娱乐 01:01:35
- **课间休息:** 老师安排休息至10点10分，并播放了歌曲。
- 3) 课程文件与工具概览 01:04:41



- - **开发环境:** 课程使用Jupyter Notebook进行代码演示，文件名为 info-extraction-agent.ipynb。
 - **编程语言:** 使用Python 3.9版本。
- 4) AI学习路径与实验训练场 01:08:44



-
- **课程体系:** 平台提供了从基础到高级的完整AI学习路径。
 - **Python Lab:** 学习AI应用开发的基础能力，如JSON处理、异步编程、并发等。
 - **Data Lab:** 掌握AI数据处理基础，包括NumPy、Pandas、数据清洗与分析。

- **Text Lab:** 学习文本处理技术，如中文分词、正则表达式、Markdown解析。
- **LLM Playground:** 探索大模型基础，包括Token、参数调优、Prompt设计、流式输出。
- **Prompt Lab:** 系统学习Prompt技术，如Zero-shot、CoT、ReAct、Reflection。
- **RAG Lab:** 学习企业级知识库系统，涵盖文档解析、向量检索、Rerank、生成。
- **Agent Lab:** 学习智能体开发。
- **Multi-Agent Lab:** 学习多智能体系统。

5) 课程继续 01:10:10

- **恢复上课:** 老师确认画面和声音正常后，课程继续进行。

3. 答疑 01:10:27

1) Pydantic数据建模 01:10:33



- **核心作用:** Pydantic是Python中最流行的数据验证库，用于定义数据模型并进行自动校验。
- **基础用法:** 通过继承BaseModel定义字段及其类型，实例化时会自动校验数据类型。
 - 例如：定义User模型，包含id: int、name: str、email: str、age: int等字段。
 - 合法数据可正常实例化，非法数据会抛出验证错误。
- **高级特性:** 支持嵌套模型定义，例如多轮对话中，每个对话可包含多个Message，每个Message又可包含多个ToolCall。
- **AI工程中的两大应用场景:**
 - **搭配大模型进行格式化输出解析:** 通过response_format (OpenAI语法) 或 tools方式，让大模型以指定格式返回数据，本质都是输出一个Pydantic model。
 - **定义配置与传参格式:** 无论是请求参数还是返回结果，都可以用Pydantic明确定义其格式和约束。

2) 调用函数写类的使用场景 01:15:07

- **使用场景:** 当输出格式不是普通文本，而是需要被程序解析的结构化格式（如JSON）时，就可以使用类来定义输出结构。

3) 关键词识别与实体识别的区别 01:15:31

- **关键词识别:** 识别句子中的关键短语，例如“我今天吃了肠粉导致拉肚子”，关键词是“肠粉”和“拉肚子”。
- **实体识别（命名实体识别）:** 识别句子中的命名实体，并为每个实体赋予一个类别标签。
 - 例如“我今天吃了肠粉导致拉肚子”中，“肠粉”是“食物”实体，“拉肚子”是“症状”实体。
 - 再如“我今天和小王以及小强去奥体吃饭，吃了大米先生”中，“小王”、“小强”是“人名”实体，“奥体”是“地理位置”实体，“大米先生”是“饭店名字”实体。

4) Tools的使用场景 01:18:30

- **Tools的两大用途:**

- **格式化解析:** 让大模型输出结构化的数据。
- **工具调用或选择:** 让大模型决定调用哪个外部工具。
- **使用与否的判断:** 如果是单纯的文本对话, 不需要使用Tools; 如果需要格式化输出或调用工具, 则需要使用Tools。

5) 作业思路总结: Pydantic model格式化 01:19:29

```
def call_tool(user_prompt, response_model):
    messages = [
        {
            "role": "user",
            "content": user_prompt
        }
    ]
    tools = [
        {
            "type": "function",
            "function": {
                "name": response_model.schema()['title'],
                "description": response_model.schema()['description'],
                "parameters": {
                    "type": "object",
                    "properties": response_model.schema()['properties'],
                    "required": response_model.schema()['required'],
                }
            }
        }
    ]
    response = client.chat.completions.create(
        model="gpt-4",
        messages=messages,
        tools=tools
    )
    try:
        arguments = response.choices[0].message.tool_calls[0].function.arguments
        return response_model.model_validate_json(arguments)
    except:
        print("ERROR", response.choices[0].message)
        return None
```

- **核心思路:** 将一个Pydantic model格式化为一个Tool JSON, 无需手动编写JSON格式。
- **实现方式:** 通过代码将Pydantic model自动转换为Tool JSON, 供大模型调用。

6) 迁移学习与泛化能力的关系 01:19:58

- **两者不等价:**
 - **迁移学习:** 是一种模型的训练方式或训练过程, 指如何复用已有模型进行训练。
 - **泛化能力:** 是模型在未见过的样本 (测试集) 上的表现能力, 指模型能否在新样本上发挥作用。

7) 最后一层全连接层与神经元数量变化 01:20:56

- **理解正确:** 最后一层通常是一个全连接层, 将10个神经元 (10分类) 变为1个神经元 (2分类), 这种理解是正确的。

8) 迁移学习与公共组件复用 01:21:07

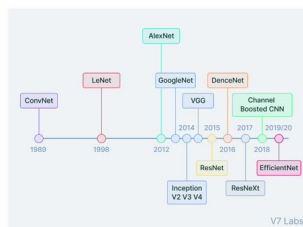
- **类似但不等同:** 迁移学习类似于“拿来主义”, 即复用已有的模型, 而不是简单地修改公共组件。

9) 传统深度学习解决方案范式 01:21:46

传统的深度学习解决方案范式

针对不同的任务设计不同的网络:

- ✓ 用于图像分类的网络
- ✓ 用于文本分类的网络
- ✓ 用于实体抽取的网络
- ✓ 用于文本翻译的网络
- ✓ ...



- **核心特点:** 针对不同的任务设计不同的网络结构, 每个数据集单独建模。
 - 例如: 图像分类网络、文本分类网络、实体抽取网络、文本翻译网络都是不同的模型。
- **局限性:** 一个任务训练的模型无法直接用于另一个任务, 因为数据集不同。

10) 迁移学习时模型的网络结构 01:22:17

- **结构变化:** 迁移学习时, 模型的网络结构基本不变, 主要修改输入层和输出层。
- **修改职责:** 针对模型这个“黑匣子”进行层级修改, 是模型算法工程师的工作。

4. 大模型应用 01:26:16

1) 大模型落地方法

	提示词工程 Prompt Engineering	检索增强生成 RAG	模型微调 Fine-tuning	预训练
优化思路	通过设计提示词优化大模型输出	通过外挂企业知识库优化大模型输出	通过行业知识内化为部分参数，优化大模型输出	全面调整或从头构建大模型
是否需要调整模型参数	不需要	不需要	需要	需要
代表技术	• 零样本提示 Zero-shot • 少样本提示 Few-shot • 思维链提示 CoT	• 文本嵌入 Text Embedding • 向量数据库	• 有监督微调 SFT • 监督微调 LORA	• 无监督学习 UL • 自监督学习 SSL • 人类反馈强化学习 RLHF

✓ 绝大部分应用中不需要微调大模型，也不需要构建领域大模型；
✓ 开发阶段可以优先使用云端大模型，最终部署从速度、成本和安全考虑可以选择本地大模型；
✓ 大模型开发的趋势是低代码和拖拽式应用，但深度开发还是偏向后端和业务开发；

- 推荐方案: 优先使用提示词工程 + 检索增强生成 (RAG) 的组合。
- 模型微调 (Fine-tuning): 虽然可行，但成本较高，不是首选方案。
- 四种方法对比:
 - 提示词工程: 通过设计提示词优化输出，不需要调整模型参数。
 - 检索增强生成 (RAG): 通过外挂企业知识库优化输出，不需要调整模型参数。
 - 模型微调: 将行业知识内化为部分参数，需要调整模型参数。
 - 预训练: 全面调整或从头构建大模型，需要调整模型参数。
- 开发建议:
 - 开发阶段可优先使用云端大模型。
 - 最终部署时，从速度、成本和安全角度考虑，可选择本地大模型。
 - 大模型开发的趋势是低代码和拖拽式应用，但深度开发仍偏向后端和业务开发。

2) Pytorch使用案例 01:27:08

- 案例1: 欺诈交易检测: 输入交易金额、时间、地点、商户类型、历史交易行为等特征，通过多层非线性变换学习复杂模式，输出交易为欺诈行为的概率。
- 案例2: 交通标志识别: 输入停车标志、限速标志等图像数据，设计模型用于自动驾驶汽车识别道路标志。
- 案例3: 人脸关键点定位: 输入人脸图像，输出眼睛、眉毛、鼻子、嘴巴等关键点坐标，用于虚拟试妆等应用。
- 模型验证过程

Pytorch使用案例

```
def evaluate(model):
    model.eval()
    correct, total = 0, 0
    with torch.no_grad():
        for data in testloader:
            inputs, labels = data
            outputs = model(inputs)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    return correct / total
```

```
Training loop
for epoch in range(5):
    model.train()
    running_loss = 0.0

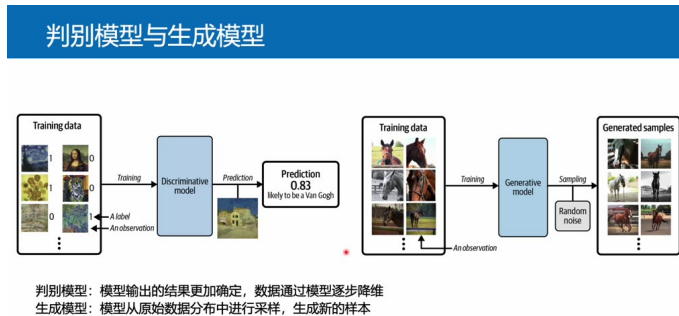
    for data in trainloader:
        inputs, labels = data
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()

    accuracy = evaluate(model)
    print(f"Epoch {epoch + 1}, Loss: {running_loss / len(trainloader)}, Accuracy: {accuracy * 100:.2f}%")
    ==>
```

- 验证方法: 计算准确率，比较模型输出结果与真实标签的差异。
- 验证步骤:
 - 设置模型为评估模式: `model.eval()`
 - 使用 `torch.no_grad()` 关闭梯度计算。
 - 遍历测试数据，获取模型输出。
 - 使用 `torch.max` 获取预测结果。
 - 统计预测正确的样本数，计算准确率: `correct / total`。
- 梯度相关问题
 - 梯度的作用: 代表模型参数权重的更新方向。

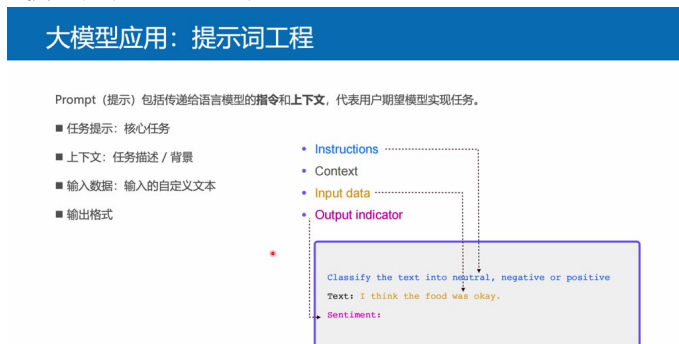
- **梯度不清空的影响**: 在PyTorch中, 梯度会**累积**, 因此每次反向传播前必须清空梯度。
- **梯度计算与参数更新**: 梯度通过反向传播计算, 用于更新模型权重。
- **超参数与权重的区别**:
 - **超参数**: 人工设定, 训练开始后**不能自动调整**。
 - **权重**: 模型内部参数, 训练过程中可以**自动调整**。

3) 判别模型与生成模型 01:27:39



-
- **判别模型**: 模型输出结果更加**确定**, 数据通过模型逐步**降维**。
- **生成模型**: 模型从原始数据分布中进行**采样**, 生成新的样本。
- **通俗理解**: 生成模型是对原始训练数据集学习其**分布**, 相当于将数据“压缩”并“记忆”下来。
- **大模型本质**: 大模型学习互联网上所有公开文本的**知识或分布**。

4) 大模型应用: 提示词工程 01:29:13

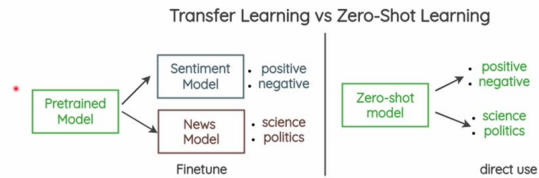


-
- **提示词 (Prompt)**: 包括传递给语言模型的指令和上下文, 代表用户期望模型完成的**任务**。
- **提示词组成**:
 - **任务提示**: 核心任务。
 - **上下文**: 任务描述或背景。
 - **输入数据**: 用户输入的自定义文本。
 - **输出格式**: 指定输出的格式要求。
- **核心优势**: 通过提示词, 可以将原有的多个单独建模的范式**统一**在一个范式中, 让大模型完成不同任务。

5) 例题1: 上下文学习与迁移学习的区别 01:30:33

大模型应用：提示词工程

上下文学习 (In-context Learning) 是大语言模型的一个关键能力。它指的是模型能够根据输入中提供的上下文信息 (比如一些示例) 来完成任任务, 而无需更新模型的参数。例如, 你给模型一个“翻译: 你好 -> Hello”的例子, 然后给它一个“翻译: 再见 -> ”的输入, 模型就能理解并完成任务。这种学习方式是GPT-3等模型的强大之处。



-
- 题目解析
 - 审题与考点: 区分上下文学习 (In-context Learning) 和迁移学习 (Transfer Learning) 的核心差异。
 - 解题思路:
 - 迁移学习: 传统深度学习的能力, 所有深度学习模型都可以做。模型在训练过程中可以“照搬”其他模型的知识, 需要调整模型参数。
 - 上下文学习: 大模型特有的能力。模型结合提示词中提供的示例进行推理, 不需要更新模型参数, 只调整输入。
 - 关键区别:
 - 迁移学习: 需要调整模型参数。
 - 上下文学习: 不需要调整模型参数, 只需要调整输入。
 - 答案: 上下文学习是大模型特有的能力, 它不需要调整模型参数, 只通过调整输入 (提示词中的示例) 来提升模型表现。

5. 语言模型 01:33:19

1) 传统语言模型 01:37:07

传统语言模型

n-gram指的是由n个连续单词构成的序列, 例如, large是一个一元语法(Unigram)的实例, large language是一个二元语法(Bigram)的实例, 而large language model是一个三元语法(Trigram)的实例。

$$P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i)$$

基于n-gram构造的语言模型称为n-gram语言模型, 这种语言模型将单词序列的生成过程看作马尔可夫过程, 其数学基础是马尔可夫假设(Markov Assumption), 即第n个词仅仅依赖于它前面的n-1个词。

$$\begin{aligned} P(\text{large, language, model}) &= P(\langle s \rangle) P(\text{large}) P(\text{language}) P(\text{model}) P(\langle s \rangle) \\ &= P(\text{large, language, model}) \\ &= P(\text{large} | \langle s \rangle) P(\text{language} | \text{large}) P(\text{model} | \text{language}) P(\langle s \rangle | \text{model}) \end{aligned}$$

n-gram会遇到未登录词(Out-Of-Vocabulary, OOV)问题, 此外当n变大时, 需要计算对应概率的n-gram的个数呈指数增长。某些n-gram在训练语料中极为稀疏, 从而导致没有足够的数据来对其概率进行准确估计, 甚至出现训练数据中未出现的n-gram被赋予零概率的情况。

-
- 语言模型 (LM) 核心任务: 对文本序列的概率分布进行建模, 本质是计算一个句子存在的概率, 或给定上文预测下文出现的概率。
- 两种等价定义:
 - 计算整个句子序列的概率: $P(w_1, w_2, \dots, w_l)$ 。
 - 计算给定上文时下一个词出现的概率: $P(w_{l+1} | w_1, \dots, w_l)$ 。
 - 等价性: 通过链式法则, 整个句子的概率可以分解为一系列条件概率的乘积。
- 应用场景:
 - 机器翻译: 选择概率更高的词序, 例如 $P(\text{the cat is small}) > P(\text{small the is cat})$ 。
 - 语音识别: 选择概率更高的同音词组合, 例如 $P(\text{there are four cats}) > P(\text{there are for cats})$ 。

传统语言模型

n-gram指的是由n个连续单词构成的序列，例如，large是一个一元语法(Unigram)的实例，large language是一个二元语法(Bigram)的实例，而large language model是一个三元语法(Trigram)的实例。

$$P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i)$$

基于n-gram构造的语言模型称为n-gram语言模型，这种语言模型将单词序列的生成过程看作马尔可夫过程，其数学基础是马尔可夫假设(Markov Assumption)，即第n个词仅仅依赖于它前面的n-1个词。

$$P(\text{large, language, model}) = P(<s>)P(\text{large})P(\text{language})P(\text{model})P(</s>)$$

$$P(\text{large, language, model}) = P(\text{large}|<s>)P(\text{language}|\text{large})P(\text{model}|\text{language})P(</s>|\text{model})$$

n-gram会遇到未登录词(Out-Of-Vocabulary, OOV)问题，此外当n变大时，需要计算对应概率的n-gram的个数呈指数增长。某些n-gram在训练语料中极为稀疏，从而导致没有足够的数据来对其概率进行准确估计，甚至出现训练数据中未出现的n-gram被赋予零概率的情况。

-
- **n-gram定义**: 由n个连续单词构成的序列。例如，large是1-gram，large language是2-gram，large language model是3-gram。
- **核心假设**: 基于马尔可夫假设，即第n个词的出现仅依赖于它前面的n-1个词，而非整个历史。
- **概率计算**: 通过链式法则，将句子概率转化为条件概率的乘积。例如，计算 $P(\text{large, language, model})$ ，可分解为 $P(\text{large}) * P(\text{language} | \text{large}) * P(\text{model} | \text{language})$ 。
- **主要问题**:
 - **未登录词 (OOV) 问题**: 训练时词表中没有的单词，在推理时无法计算概率。
 - **维数灾难**: 当n增大时，可能的n-gram组合数量呈指数级增长（例如5万词表，5-gram组合数为5万的5次方），导致模型参数爆炸，无法存储和计算。
 - **数据稀疏性**: 许多n-gram在训练语料中极少出现，导致概率估计不准确，甚至出现零概率。

2) 神经网络语言模型 01:41:24

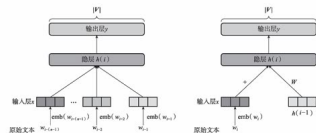
神经网络语言模型

神经网络语言模型(Neural Network Language Model, NNLM)指的是一类利用神经网络分类器来计算某个上下文中的单词或单词序列的概率的语言模型。神经网络语言模型依赖于词嵌入(Word Embedding)和多层神经网络结构来完成上述概率的计算。

神经网络语言模型可以对可能出现在同一上下文中的其他词的词嵌入进行加权组合，并以此作为未登录词的概率。神经网络语言模型并不需要显式存储每个n-gram及其对应的概率，因而可以有效减轻n-gram语言模型中的维数灾难问题。

✓ 前馈神经网络语言模型中，将上下文中的每个单词映射为其对应的词嵌入，然后将这些词嵌入拼接起来作为输入，通过Softmax函数最终输出一个在词汇表上的概率分布。

✓ 循环神经网络语言模型中每个隐层依赖于当前单词的词嵌入和循环神经网络中上一个状态的隐层，通过这样的机制，循环神经网络语言模型便能利用任意长度的上下文信息。



-
- **NNLM定义**: 利用神经网络分类器来计算单词或单词序列概率的语言模型。
- **核心依赖**: 依赖于词嵌入 (Word Embedding) 和多层神经网络结构。
- **工作原理**:
 - 将上下文中的每个单词映射为对应的词嵌入向量。
 - 将这些词嵌入向量拼接或加权求和，作为神经网络的输入。
 - 通过Softmax函数，最终输出一个在词汇表上的概率分布，本质是一个多分类任务。
- **优势**:
 - **缓解OOV问题**: 可以通过对上下文中其他词的词嵌入进行加权组合，来估算未登录词的概率。
 - **缓解维数灾难**: 不需要显式存储每个n-gram及其概率，而是通过神经网络学习参数，有效减轻了数据稀疏问题。
- **局限性**: 模型参数量仍然较大，且模型复杂度有限。

3) 词向量基础 01:45:24

词向量基础

□词的一种机内表示形式，便于计算

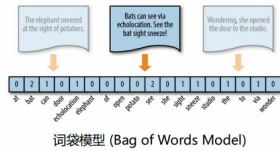
□传统使用one-hot词向量表示词

- 高维、稀疏、离散
- 导致严重的数据稀疏问题
- 所有向量都是正交的

土豆 [0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, ...]

马铃薯 [0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, ...]

$\text{sim}(\text{土豆}, \text{马铃薯}) = 0$ 😞



-
- 词向量 (Word Embedding) : 将词转换为适合神经网络计算的数值表示形式。
- 传统方法: One-hot编码:
 - 用一个非常长且稀疏的向量表示词，向量长度等于词表大小。
 - 每个词对应向量中唯一的一个维度为1，其余为0。
 - 缺点: 高维、稀疏、离散，且所有向量都是正交的，无法表示词与词之间的语义相似度（例如“土豆”和“马铃薯”的向量点积为0）。
- 词袋模型 (Bag of Words) :
 - 本质上是One-hot编码的扩展，用于表示一个句子或文档。
 - 统计句子中每个词出现的频次，用频次向量来表示整个句子。
 - 缺点: 仍然基于One-hot表示，无法捕捉词序和语义信息。

二、词向量基础 01:50:16

1. 词的一种机内表示形式，便于计算 01:50:21

1) 传统使用one-hot词向量表示词 01:50:24

词向量基础

□词的一种机内表示形式，便于计算

□传统使用one-hot词向量表示词

- 高维、稀疏、离散
- 导致严重的数据稀疏问题
- 所有向量都是正交的

土豆 [0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, ...]

马铃薯 [0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, ...]

$\text{sim}(\text{土豆}, \text{马铃薯}) = 0$ 😞

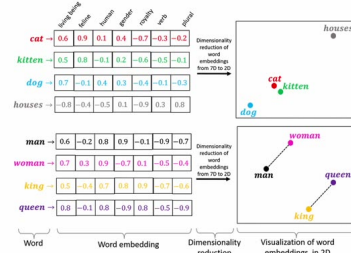


- - 核心问题: 传统词表示方法（如词袋模型、one-hot编码）存在严重缺陷。
 - one-hot编码特点:
 - 高维、稀疏、离散: 向量的维度等于词表大小（如5万维），每个词向量只有一个位置是1，其余全是0。
 - 数据稀疏: 导致严重的数据稀疏问题。
 - 向量正交: 所有向量之间夹角都是90度，相似度恒为0。
 - 根本缺陷: 无法计算语义相似度。例如，“土豆”和“马铃薯”的one-hot向量完全不同，无法体现它们是同义词。
- 2) 分布表示 01:52:53

词向量基础

分布语义假设 (Distributional semantic hypothesis)

词的含义可由其上下文词的分布进行表示



- **核心思想: 分布语义假设:** 词的含义可由其上下文词的分布进行表示。
- **分布式表示方法:**
 - 不再用单个维度表示词是否出现, 而是用**多个维度** (如7维、100维) 的**浮点数**来表示词的含义。
 - 每个维度代表词在该“语义特征”上的强度, 无需显式定义每个维度的具体含义。
- **优势:** 词与词之间不再是正交关系, 可以在多维空间中计算相似度, 语义相近的词距离更近。
- **分布表示的缺点 01:54:07**
 - **训练速度慢:** 需要单独训练一个编码模型。
 - **扩展困难:**
 - 增加新语料库或扩充词表时, 需要重新训练模型。
 - 不易扩展到短语、句子的表示。
- **分布表示的优点 01:55:01**
 - **有效语义表示:** 能获得更有效的语义相似度表示, 原始含义相近的词, 编码后距离依然相近。
 - **低维、稠密、连续:**
 - **低维:** 维度固定 (如100、300、4096维), 远低于one-hot的词表大小 (如5万维)。
 - **稠密:** 向量中的每个位置都是非零的浮点数, 可正可负。
 - **连续:** 通过浮点数表示, 数值是连续的。
 - **自监督学习:** 可以直接通过自监督的方式训练, 无需人工标注。
- **分布表示的发展 01:55:18**

词向量基础

分布表示的缺点

训练速度慢, 增加新语料库困难

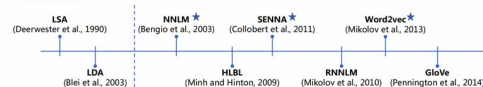
不易扩展到短语、句子表示

分布式表示直接使用低维、稠密、连续的向量表示词

通过“自监督”的方法直接学习词向量

也称词嵌入 (Word Embedding)

发展历程



- **发展历程:** LSA (1990) → NNLM (2003) → SENNA (2011) → Word2Vec (2013) → GloVe (2014)。
 - **后续影响:** 后续的BERT、GPT等大模型也参考了类似的分布式表示思想。
- ## 2. 词嵌入 01:57:07
- **定义:** 将原始词编码成固定维度的向量, 这个过程称为**词嵌入**。
 - **核心作用:**
 - 每个单词编码成一个固定维度的向量。

- 可以在向量空间中计算文本与文本之间的相似度。
- 可以进一步用于文本建模。

3. 词向量的发展 01:57:44

1) 发展历程 01:57:46

- **基础概念:** 词向量的发展通常与语言模型相结合，这部分知识主要用于面试，实际项目中已较少直接使用。

2) Word2Vec 01:58:01

Word2Vec

Word2Vec通过学习文本语料库中的词语上下文关系，将每个词语表示为一个密集的向量，使得语义相似的词在向量空间中距离较近。

Word2Vec主要有两种模型结构：Skip-gram和CBOW (Continuous Bag of Words)。

- ✓ **Skip-gram (跳字模型)**：给定一个中心词，Skip-gram模型试图预测在其周围上下文中可能出现的其他词语。
- ✓ **BOW (连续词袋模型)**：CBOW模型与Skip-gram相反，它试图通过上下文中的多个词来预测中心词。

CBOW模型示意图

Skip-gram模型示意图

Window Size	Text	Skip-grams
2	[The wide road shimmered] in the hot sun.	wide, the wide, road wide, shimmered
	The [wide road shimmered] in the hot sun.	shimmered, wide shimmered, road shimmered, in shimmered, the
	The wide road shimmered in [the hot sun].	sun, the sun, hot
3	[The wide road shimmered in] the hot sun.	wide, the wide, road wide, shimmered wide, in
	shimmered, the shimmered, wide shimmered, road shimmered, in shimmered, the	shimmered, hot
	The wide road shimmered [in the hot sun].	sun, in sun, the sun, hot

- **核心思想:** 通过学习文本语料库中词语的上下文关系，将每个词表示为稠密向量，使语义相似的词在向量空间中距离较近。
- **两种模型结构:**
 - **CBOW (连续词袋模型):** 通过上下文中的多个词来预测中心词。
 - **Skip-gram (跳字模型):** 给定中心词，预测其周围上下文中可能出现的词。
- **训练方式:** 自监督学习，无需额外标注。
- **与NNLM的关系:** 本质与NNLM非常相似，但Word2Vec加入了负采样和Softmax优化等训练细节，性能更优。

3) FastText 01:59:55

FastText

FastText引入了n-gram的概念，将单词表示为其字符n-gram的平均值的总和。这有助于捕捉单词内的局部结构，尤其是对形态学丰富的语言。

FastText使用一个简单的多层感知机作为神经网络结构，这有助于在训练时更好地学习上下文信息。FastText采用了分层的Softmax方法，它在训练时可以高效地处理大规模词汇。

actual context

am * food

eating

0.7 0.4

bring to 1

negative context

paris earth

0.5 0.3

bring to 0

dot product

sigmoid

goal

- **核心改进:** 引入**n-gram**概念，将单词表示为多个字符n-gram嵌入结果的平均值。
- **优势:** 能捕捉单词内部的局部结构，尤其适用于形态学丰富的语言。
- **训练过程:** 与Word2Vec非常相似，只是输入表示方式不同。

4) GloVe 02:00:45

GloVe

GloVe (Global Vectors for Word Representation) 是一种用于学习词向量的无监督算法，由斯坦福大学的研究团队开发。GloVe的设计目标是在全局范围内捕获词语之间的语义关系，通过将词语表示为向量来实现这一目标。

GloVe使用了文本中词语的共现矩阵，该矩阵记录了词语对在同一上下文中出现的频率。这有助于模型了解词语之间的关系。GloVe的核心思想是通过点乘运算 (element-wise product) 来构建损失函数，从而使得学到的词向量满足具有实际意义的线性关系。

- 利用“词-上下文”共现信息
 - Word2Vec: 局部共现，只考虑当前样本中是否共现
 - GloVe: 利用全局统计信息，即共现频次
- 利用词向量对“词-上下文”共现矩阵进行预测 (或回归)
 - 构建共现矩阵: 共现“强度”按照距离进行衰减

$$M_{w,c} = \sum_i \frac{1}{d_i(w,c)}$$

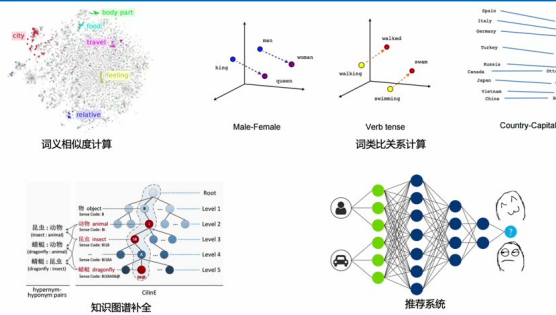
- 回归目标: $v_w^T v_c' + b_w + b_c' = \log M_{w,c}$

- 参数估计 $\mathcal{L}(\theta; M) = \sum_{(w,c) \in \mathcal{D}} f(M_{w,c})(v_w^T v_c' + b_w + b_c' - \log M_{w,c})^2$

-
- **核心思想:** 利用全局词-词共现信息进行无监督学习。
- **与Word2Vec的区别:**
 - **Word2Vec:** 局部共现，只考虑当前样本中是否共现。
 - **GloVe:** 利用全局统计信息 (共现频次)，对词-上下文共现矩阵进行回归预测。
- **训练目标:** 期望两个词向量的点积与它们在整个语料库中的共现次数尽可能一致，可理解为回归任务。

4. 词向量的应用 02:02:51

词向量的应用



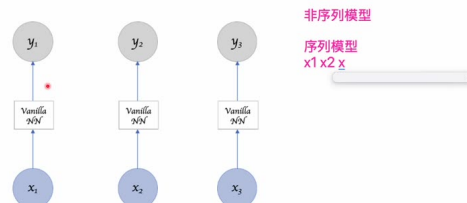
-
- **核心用途:** 将单词转换为向量，用于后续计算。
- **具体应用场景:**
 - **词义相似度计算:** 计算文本与文本之间的相似度。
 - **词类比关系计算:** 如“国王-王后”类比于“男人-女人”。
 - **知识图谱补全:** 用于推理实体间的关系。
 - **推荐系统:** 将用户或物品表示为向量进行推荐。

三、判别模型与生成模型 02:04:07

1. 非序列模型 02:04:31

判别模型与生成模型

假设每次输入都是独立的，也就是说每次网络的输出只依赖于当前的输入。



思考: 什么场景下, 当前结果的输出与之前的输入有关?

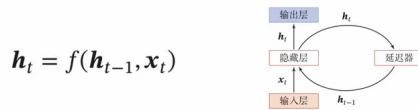
-
- **核心概念:** 模型的每次输出只依赖于当前的输入，与之前的输入无关。
 - 这种模型被称为**非序列模型**。

- 例如：输入 x_1 得到 y_1 ，输入 x_2 得到 y_2 ，输入 x_3 得到 y_3 ，这些过程相互独立。
- **思考题:** 什么场景下，当前结果的输出与之前的输入有关？
 - 当输入是一个序列时，例如文本或句子，当前输出往往依赖于历史信息。

2. 深度学习中的序列层 02:06:14

深度学习（常见网络层）

循环神经网络通过使用带自反馈的神经元，能够处理任意长度的时序数据。



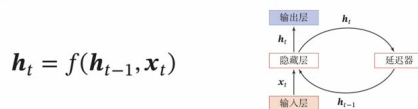
- ✓ 循环神经网络比前馈神经网络更加符合生物神经网络的结构。
- ✓ 循环神经网络已经被广泛应用于语音识别、语言模型以及自然语言生成等任务上

-
- **序列模型:** 用于处理序列输入（如文本、语音）的模型。
 - 输入可以是一个序列，例如 x_1, x_2, x_3 。
- **常见序列层:** 深度学习中的序列层主要包括RNN、LSTM和GRU。
 - 这些层是构建大语言模型的基础，在面试中常被考察。

3. 循环神经网络 02:07:02

深度学习（常见网络层）

循环神经网络通过使用带自反馈的神经元，能够处理任意长度的时序数据。



- ✓ 循环神经网络比前馈神经网络更加符合生物神经网络的结构。
- ✓ 循环神经网络已经被广泛应用于语音识别、语言模型以及自然语言生成等任务上

-
- **核心概念:** 循环神经网络（RNN）通过使用带自反馈的神经元，能够处理任意长度的时序数据。
 - **自反馈机制:** 隐藏层可以保留历史信息，并在下一个时刻使用。
 - **核心公式:** $h_t = f(h_{t-1}, x_t)$ ，其中 h_t 是当前隐藏状态， h_{t-1} 是上一时刻的隐藏状态， x_t 是当前输入。
- **与生物神经网络的相似性:** RNN比前馈神经网络更符合生物神经网络的结构。
- **应用场景:** 广泛应用于语音识别、语言模型和自然语言生成等任务。
- **工作流程:** 输入一个序列，模型会输出一个对应的序列，每个时刻的输入和输出一一对应。
 - 隐藏状态 h 会随着时间推移不断保存和更新记忆，使得处理当前输入 x_t 时，已经包含了之前的信息。

4. 循环神经网络的问题 02:10:15

深度学习中的序列层

由于梯度爆炸或消失，实际上RNN只能学习到短周期的依赖关系，这就是所谓的长程依赖问题。

- 梯度爆炸问题
 - 权重衰减
 - 梯度截断
- 梯度消失问题
 - 改进模型

$$\frac{\partial \mathcal{L}}{\partial U} = \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k} \mathbf{h}_{k-1}^T$$

-
- **长程依赖问题**: 由于梯度爆炸或消失，RNN实际上只能学习到短周期的依赖关系。
 - **梯度爆炸**: 梯度值变得非常大（如大于100），导致模型参数更新不稳定。
 - **解决方法**: 权重衰减、梯度截断。
 - **梯度消失**: 梯度值变得非常小（如接近0），导致模型无法学习长距离依赖。
 - **解决方法**: 改进模型结构（如LSTM、GRU）。
- **原因**: RNN在反向传播时，需要逐层计算梯度，存在逐层相乘的过程，导致最终梯度要么太大，要么太小。

5. 问题回答 02:11:33

- **梯度下降的本质**: 调整模型的权重和偏置参数，使损失函数最小化。
 - **判断模型合理性**: 通过评价指标（如准确率、困惑度）来衡量。
- **过拟合判断**: 通过划分验证集，观察模型在验证集上的效果来判断是否过拟合。
- **微调与避免微调**: 在大模型应用中，通常避免微调，因为微调可能导致模型能力退化或知识遗忘。
- **语言模型与语法错误**: 大语言模型不能保证完全无语法错误，它是从语料中学习如何表达，逐步输出token。
- **NNLM与深度学习的关系**: NNLM是一个深度学习模型，专门用于语言建模任务。
- **词向量化**: 可以理解为将一个单词转换成一个数字向量。
- **sin的含义**: 在课件中，sin代表相似度计算（similarity）。
- **词袋模型**: 词袋模型（BoW）没有解决“一词多义”等问题，它只是进行词频统计。

四、神经网络语言模型 02:16:31

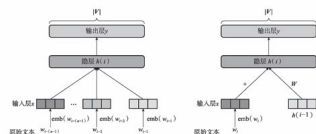
神经网络语言模型

神经网络语言模型(Neural Network Language Model, NNLM)指的是一类利用神经网络分类器来计算某个上下文中的单词或单词序列的概率的语言模型。神经网络语言模型依赖于词嵌入(Word Embedding)和多层神经网络结构来完成上述概率的计算。

神经网络语言模型可以对可能出现在同一上下文中的其他词的词嵌入进行加权组合，并以此作为未登录词的词嵌入计算未登录词的概率。另外，神经网络语言模型并不需要显式存储每个n-gram及其对应的概率，因而可以有效减轻n-gram语言模型中的维数灾难问题。

✓ 前馈神经网络语言模型中，将上下文中的每个单词映射为其对应的词嵌入，然后将这些词嵌入拼接起来作为输入，通过Softmax函数最终输出一个在词汇表上的概率分布。

✓ 循环神经网络语言模型中每个隐层依赖于当前单词的词嵌入和循环神经网络中上一个状态的隐层，通过这样的机制，循环神经网络语言模型便能利用任意长度的上下文信息。



-
- **核心定义**: 神经网络语言模型（NNLM）利用神经网络分类器，计算某个上下文中的单词或单词序列的概率。
- **关键依赖**: 依赖于词嵌入（Word Embedding）和多层神经网络结构来完成概率计算。
- **前馈NNLM**: 将上下文中的每个单词映射为词嵌入，拼接后作为输入，通过Softmax函数输出词汇表上的概率分布。
- **循环NNLM**: 每个隐层依赖于当前单词的词嵌入和上一状态的隐层，因此能利用任意长度的上下文信息。

- **核心优势:**
 - 可以对同一上下文中其他词的词嵌入进行加权组合，以此计算未登录词的概率。
 - 不需要显式存储每个n-gram及其概率，有效减轻了n-gram语言模型中的维数灾难问题。

五、问题回答 02:18:02

1. www、ron的实际应用 02:18:03

- **www:** 面试时会问到，但在实际工作中基本用不到。
- **ron:** 讲解ron的缺点只是为了知识点完备和应对面试题，现在实际上没有直接使用。

2. 梯度消失与梯度爆炸 02:18:41

- **理解方式:** 反向传播的梯度是乘法运算，这样理解是没问题的。

3. 多维词向量 02:18:58

- **解释:** 多维词向量的具体实现方式会在下午的课程中解释。

4. 作业借助AI写代码 02:19:13

- **老师观点:** 能用AI写出代码也是你的能力，代表你掌握了相关知识。
- **建议:** 最好还是自己手写一遍，把知识点串起来会更好。

5. bge模型 02:19:42

- **bge与词嵌入的关系:** bge是一个Bert模型，Bert模型的输入使用了词嵌入（Word Embedding）。
- **bge的功能:** bge是做句子嵌入（Sentence Embedding），把一个句子转变成一个向量，不是单纯的词嵌入。

6. 神经元随机选择与网络恢复 02:19:56

- **选择方式:** Dropout是随机选择放弃哪些神经元。
- **恢复机制:** 训练时（如50%节点被抑制）信号较弱，实际使用完整网络时信号更强。因此，实际使用时会对输入信息进行缩放，以保证输出强度与训练时一致。

7. 深度学习调参：学习率 02:20:15

深度学习调参

学习率决定了网络优化速度，也决定了跳出局部最优的可能性。

- 动态学习率
- 自适应学习率

标准的（小批量）梯度下降
学习率

$$\Delta \theta_t = -\frac{\alpha}{\sqrt{G_t + \epsilon}} \odot \mathbf{g}_t$$

✓ 学习率衰减

✓ Adagrad

✓ Adadelta

✓ RMSprop

梯度

✓ Momentum

✓ Nesterov accelerated gradient

✓ 梯度截断

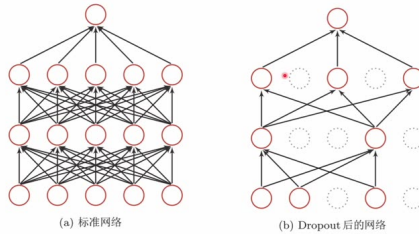
-
- **学习率的作用:** 决定网络优化速度，也决定了跳出局部最优的可能性。
- **常见方法:**
 - 动态学习率
 - 标准（小批量）梯度下降
 - 自适应学习率（如Adagrad、Adadelta、RMSprop）
 - 学习率衰减
- **梯度相关:** 动量（Momentum）、Nesterov加速梯度、梯度截断。

8. 深度学习调参：Dropout 02:20:25

深度学习调参

Dropout

- 集成学习：原始网络可以近似看作是不同的子网络的组合模型。
- 贝叶斯学习：参数 θ 为随机向量，并且先验分布为 $q(\theta)$



- 集成学习视角：原始网络可以近似看作是不同的子网络的组合模型。
- 贝叶斯学习视角：参数 θ 为随机向量，且先验分布为 $q(\theta)$ 。

9. 判别模型、生成模型与序列模型的关系 02:21:29

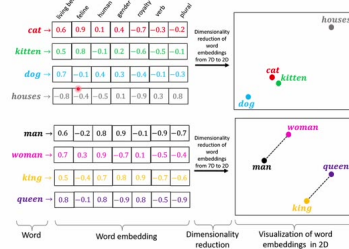
- 不同维度的划分：判别模型/生成模型、有监督模型/无监督模型/强化学习模型、非序列模型/序列模型，是从不同维度对模型进行划分。
- 序列模型的定义：能够接收序列输入并输出序列的模型称为序列模型。

10. 例题1:词向量基础与分布语义假设 02:22:49

词向量基础

分布语义假设 (Distributional semantic hypothesis)

词的含义可由其上下文词的分布进行表示



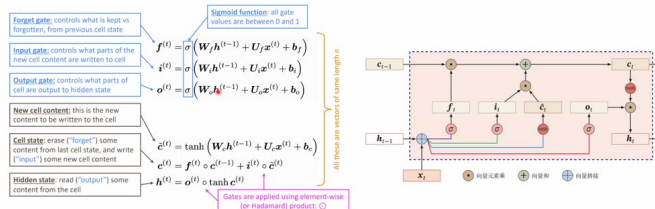
题目解析

- 审题与考点：考察词向量的核心概念、分布语义假设以及词向量的应用。
- 解题思路/思路：
 - 分布语义假设：词的含义可由其上下文词的分布进行表示。
 - 词向量降维：高维词向量可通过降维（如7D到2D）进行可视化。
 - 词向量应用：包括词义相似度计算、词类比关系计算（如Male-Female、Verb tense、Country-Capital）、知识图谱补全、推荐系统等。
- 答案：无具体题目，为知识点讲解。

11. 深度学习中的序列层：LSTM 02:23:03

深度学习中的序列层

长短期记忆神经网络 (Long Short-Term Memory, LSTM)



- LSTM核心机制：通过门控机制控制信息流动。
- 遗忘门：控制从上一细胞状态中遗忘哪些信息。

- **输入门**: 控制将哪些新细胞内容写入细胞状态。
- **输出门**: 控制将细胞状态的哪些部分输出到隐藏状态。
- **新细胞内容**: 通过tanh函数生成待写入细胞的新内容。
- **细胞状态更新**: 遗忘部分旧内容, 写入部分新内容。
- **隐藏状态更新**: 从细胞状态中读取部分内容输出。

12. 作业代码与学习建议 02:23:24

- **作业代码获取**: 上周的作业代码已发给班主任老师, 可以联系班主任转发。
- **零基础学习建议**: 跟着课程学习即可。
- **代码能力**: 能看懂代码、能web coding就可以, 如果能自己写会更好。

13. 模型划分与向量相似度计算 02:23:40

- **模型划分**: 对同一个事物(模型)可以从多个方面(如判别/生成、有监督/无监督、序列/非序列)进行归纳, 得到不同的结论。
- **向量相似度计算**: 可以通过余弦相似度进行计算。

14. 分布式语义假设与微调知识注入 02:24:20

- **分布式语义假设**: 词在不同维度的数值是通过训练得到的。
- **微调与知识注入**: 微调可以注入知识。

六、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
深度学习基础	通过多层网络逐层计算得到输出; 参数(权重)可自动调整, 超参数(如模型结构、学习率)需人工定义	参数 vs 超参数: 参数是模型权重, 超参数是人工设定的结构/训练配置	★★
迁移学习	将已有模型(含网络结构和权重)复用到新任务, 仅修改输出层; 微调(Fine-tuning)是在原有参数基础上继续训练	迁移学习 vs 上下文学习: 迁移学习需调整模型参数, 上下文学习仅调整输入提示词	★★★
模型调参	关键超参数: epoch(完整数据集迭代次数)、batch size(批量样本数)、learning rate(学习率); 通过loss曲线观察训练效果	epoch vs batch: 一个epoch包含多个batch; 学习率越大参数调整越快	★★
Dropout层	训练时随机关闭部分节点(如30%), 用残缺网络训练; 推理时用完整网络	训练时随机关闭, 推理时完整使用; 需对输出信号做缩放补偿	★★

判别模型 vs 生成模型	判别模型做判断/降维（如文本分类）；生成模型生成新样本（如GPT、千问），维度不降低	判别模型输出是类别标签，生成模型输出是完整样本；大语言模型本质是生成模型	★★★
词向量 (Word Embedding)	将单词编码为低维稠密连续向量（如100/300维），可计算语义相似度；分布式表示优于One-hot	One-hot是稀疏高维正交表示，无法计算相似度；词向量通过训练得到，语义相近的词向量距离近	★★★
N-gram语言模型	基于马尔可夫假设，统计n个单词组合的概率；n通常取3，过大导致参数指数爆炸	存在OOV（未登录词）问题；参数随n增大呈指数增长	★★
循环神经网络 (RNN)	专为序列输入设计，有自反馈机制可保留历史信息；存在梯度消失/爆炸问题	RNN vs 全连接网络：RNN可处理任意长度序列，但训练时梯度易消失或爆炸	★★★
上下文学习 (In-Context Learning)	大模型特有功能：通过提示词中的示例样本提升推理效果，不修改模型参数	上下文学习 vs 迁移学习：上下文学习不调整模型，仅调整输入；迁移学习需调整模型参数	★★
PyDantic数据模型	用于定义数据模型和格式校验；可自动生成JSON Schema，用于构建Tool的JSON格式	可搭配大模型使用：定义response format或tool schema，国产模型不支持response format但可通	★★

		过tool方式实现	
Tool（工具）定义	工具需定义名称、描述、参数（含必选/可选）；大模型根据名称和描述选择工具	工具定义类似Skill，但工具需详细定义参数格式；可通过PyDantic自动生成	★★
信息抽取与知识图谱	通过定义数据模型（如source person、target person、relationship），让大模型按格式抽取实体关系	关键词识别 vs 实体识别：关键词是提取关键词短语，实体识别是识别命名实体（如人名、地点）	★★★